

Hướng dẫn Bài tập lớn

Chủ đề, phạm vi và quy trình thực hiện

ThS. Vũ Quang Dũng

Khoa Hệ thống thông tin – Trường Đại học Phenikaa

CSE703093 – An toàn phần mềm

- 1 Tổng quan về Bài tập lớn
- 2 Danh sách chủ đề gợi ý
- 3 Quy tắc và Yêu cầu chung

Vị trí của Bài tập lớn trong đánh giá học phần

- **KTHP - Bài tập lớn (không thuyết trình): 50% điểm học phần**
- Đánh giá theo chuẩn đầu ra: **LO.03** (đánh giá kỹ thuật thẩm định/kiểm chứng) và **LO.04** (thực hiện giải quyết lỗi hỏng bằng kiểm chứng/kiểm thử)
- Đây là hình thức đánh giá **tổng hợp** toàn bộ 4 chương đã học, không phải chỉ một chương đơn lẻ

Mục tiêu của Bài tập lớn

- Áp dụng *thực hành* các kỹ thuật đã học vào một hệ thống/chương trình cụ thể (không chỉ dừng ở lý thuyết)
- Thể hiện khả năng **kết hợp** nhiều kỹ thuật: mô hình hóa (Chương 1) → tìm lỗi cụ thể (Chương 2) → kiểm chứng hình thức (Chương 3) → kiểm thử động (Chương 4)
- Rèn luyện kỹ năng viết báo cáo kỹ thuật rõ ràng, có bằng chứng cụ thể (không chỉ khẳng định suông)

Phạm vi phù hợp: Chọn "vừa sức"

Lưu ý

Bài tập lớn **không yêu cầu** phân tích một hệ thống khổng lồ. Một chương trình/module nhỏ (100–500 dòng mã) được phân tích *sâu và đầy đủ* có giá trị hơn nhiều một hệ thống lớn chỉ được phân tích hời hợt.

Nguyên tắc chọn phạm vi: đủ nhỏ để hiểu *toàn bộ* logic trong thời gian cho phép, đủ phức tạp để có ít nhất 2-3 loại lỗi/thuộc tính đáng phân tích.

Timeline gợi ý (giả định 3-4 tuần thực hiện)

Tuần 1: Chọn đề tài, chọn/viết chương trình mục tiêu, mô hình hóa ban đầu



Tuần 2: Áp dụng kỹ thuật Chương 2-3 (static analysis, BMC nếu phù hợp)



Tuần 3: Áp dụng kỹ thuật Chương 4 (fuzzing), thu thập kết quả



Tuần 4: Viết báo cáo, rà soát, hoàn thiện mã nguồn nộp kèm

- Toàn bộ 4 module (ch1_models, ch2_memsafe, ch3_staticsat, ch4_dynafuzz) đã xây dựng xuyên suốt học phần **có thể và nên được tái sử dụng trực tiếp**
- Không bắt buộc dùng toàn bộ 4 module - chọn module phù hợp với đề tài đã chọn
- Được phép mở rộng/tùy biến module cho phù hợp với chương trình mục tiêu cụ thể (khuyến khích, không bắt buộc)

Tổng quan 5 hướng đề tài

- ➊ Mô hình hóa & Kiểm chứng hình thức (Kripke + BMC)
- ➋ Kiểm toán an toàn bộ nhớ C (Memory Safety Audit)
- ➌ Kiểm chứng thuộc tính số học/alignment bằng SMT
- ➍ Dự án Fuzzing (Black-box vs White-box)
- ➎ Quy trình end-to-end tích hợp cả 4 chương

Sinh viên có thể chọn 1 trong 5 hướng, hoặc đề xuất hướng khác (cần trao đổi trước với giảng viên).

Chủ đề 1: Mô hình hóa & Kiểm chứng hình thức

- **Mô tả:** Chọn một hệ thống trạng thái đơn giản (máy ATM, hệ thống đăng nhập, đèn giao thông thông minh, máy bán hàng tự động, cửa cuốn tự động...)
- **Yêu cầu:** Xây dựng Kripke structure đầy đủ, viết ít nhất 3 thuộc tính LTL/CTL (safety + liveness), chạy BMC (dùng `ch3_staticsat.bounded_checker`) tìm completeness threshold
- **Độ khó:** Trung bình – phù hợp sinh viên thích tư duy hình thức/toán học

Chủ đề 1: Ví dụ cụ thể (gợi ý mở rộng)

Hệ thống khóa cửa thông minh với các trạng thái: locked, unlocking, unlocked, alarm.

- Thuộc tính safety: "Không bao giờ ở trạng thái unlocked mà không qua unlocking"
- Thuộc tính liveness: "Sau khi nhập đúng mã, cuối cùng sẽ mở khóa"
- Thử nghiệm: cố tình thiết kế sai (bỏ qua bước xác thực) để BMC tìm ra counterexample, sau đó sửa và xác nhận lại

Chủ đề 2: Kiểm toán an toàn bộ nhớ C

- **Mô tả:** Chọn một chương trình C nhỏ có sẵn (tự viết hoặc từ mã nguồn mở nhỏ) xử lý input từ file/mạng
- **Yêu cầu:** Chạy `ch2_memsafe.pattern_scanner`, `cppcheck`, biên dịch với ASan/UBSan; phát hiện tối thiểu 2 lỗi bộ nhớ thực sự (có thể tự chèn nếu chương trình quá sạch); sửa và xác nhận lại
- **Độ khó:** Trung bình-Thấp - phù hợp sinh viên thích lập trình C thực hành

Chủ đề 2: Ví dụ cụ thể (gợi ý mở rộng)

Chương trình phân tích file cấu hình dạng key=value đơn giản.

- Cố ý cài lỗi: dùng strcpy không giới hạn khi đọc value, quên kiểm tra NULL sau malloc
- Chạy static analysis phát hiện; xác nhận bằng cách chạy chương trình với input dài gây crash thực tế
- Sửa bằng snprintf, kiểm tra NULL; tính lại kích thước struct với alignment tối ưu nếu có dùng struct

Chủ đề 3: Kiểm chứng bằng SMT

- **Mô tả:** Chọn 2-3 hàm C nhỏ có phép toán số học hoặc truy cập bộ nhớ qua con trỏ ép kiểu
- **Yêu cầu:** Dùng `ch3_staticsat.smt_wrapper` viết ràng buộc Z3 chứng minh/tìm phản ví dụ cho: tràn số nguyên, vi phạm alignment, hoặc điều kiện an toàn tùy chỉnh
- **Độ khó:** Cao – phù hợp sinh viên thích toán học/logic hình thức

Chủ đề 3: Ví dụ cụ thể (gợi ý mở rộng)

Hàm tính chỉ số mảng từ 2 tham số đầu vào: $\text{index} = \text{base} + \text{offset} * \text{stride}$.

- Chứng minh (hoặc tìm phản ví dụ): với mọi base , offset , stride trong khoảng hợp lệ đã định nghĩa, index luôn nằm trong phạm vi mảng
- Nếu tìm được phản ví dụ (SAT), đề xuất ràng buộc bổ sung ở tầng input validation để loại trừ trường hợp đó

Chủ đề 4: Dự án Fuzzing

- **Mô tả:** Chọn một hàm/chương trình phân tích cú pháp (parser) có ít nhất một điều kiện dạng "magic byte" hoặc checksum
- **Yêu cầu:** Dùng ch4_dynafuzz chạy cả black-box và white-box fuzzing, so sánh định lượng (số lần thử, thời gian) để tìm ra cùng một lỗi/nhánh mục tiêu
- **Độ khó:** Trung bình-Cao – phù hợp sinh viên thích thực nghiệm định lượng

Chủ đề 4: Ví dụ cụ thể (gợi ý mở rộng)

Parser cho định dạng file tự thiết kế: 4 byte magic number + 4 byte độ dài + payload.

- Black-box fuzzing: đo số lần thử trung bình để tìm đúng magic number
- White-box fuzzing: dùng Z3 giải trực tiếp ra magic number, đo thời gian
- So sánh và giải thích khác biệt bằng lý thuyết đã học (Chương 4, mục 4.6)

Chủ đề 5: Quy trình End-to-end tích hợp

- **Mô tả:** Chọn MỘT chương trình duy nhất, áp dụng *tuần tự* cả 4 module của toolkit lên chương trình đó
- **Yêu cầu:** Mô hình hóa (ch1) → static analysis + sửa lỗi bộ nhớ (ch2) → kiểm chứng SMT/BMC cho 1 thuộc tính quan trọng (ch3) → fuzzing xác nhận (ch4)
- **Độ khó:** Cao nhất (khối lượng công việc lớn hơn) nhưng thể hiện rõ nhất tư duy tổng hợp toàn học phần – **khuyến nghị** cho sinh viên muốn đạt điểm cao

So sánh 5 chủ đề: Độ khó và Khối lượng

Chủ đề	Độ khó	Khối lượng
1. Kripke + BMC	Trung bình	Vừa phải
2. Memory Safety Audit	Trung bình-Thấp	Thấp-Vừa
3. SMT Verification	Cao	Vừa phải
4. Fuzzing Project	Trung bình-Cao	Vừa phải
5. End-to-end tích hợp	Cao nhất	Cao nhất

Yêu cầu tối thiểu cho mọi đề tài

- Mô tả rõ ràng chương trình/hệ thống mục tiêu (đủ chi tiết để người đọc hiểu mà không cần xem mã nguồn trước)
- Áp dụng ít nhất **2 chương** trong số 4 chương đã học (không tính Chương 1 nếu chỉ dùng làm nền tảng khái niệm)
- Có **bằng chứng cụ thể** (log, screenshot, kết quả chạy) cho mọi kết luận đưa ra
- Mã nguồn đầy đủ, chạy được, kèm hướng dẫn cài đặt/chạy lại

Lưu ý

- Sao chép một chương trình có sẵn từ mạng mà không hiểu logic bên trong – không thể phân tích sâu nếu không hiểu chương trình
- Chỉ chạy công cụ và dán kết quả mà không giải thích ý nghĩa
- Chọn chương trình quá lớn/phức tạp khiến không đủ thời gian phân tích đầy đủ
- Chỉ dùng 1 kỹ thuật duy nhất mà không kết nối với các chương khác đã học

Được phép và khuyến khích

- Viết chương trình mục tiêu **tự tay** (đơn giản hơn nhưng hiểu rõ 100%) thay vì lấy từ nguồn ngoài
- Cố ý cài lỗi vào chương trình tự viết để có "đối tượng" rõ ràng cho công cụ phát hiện (miễn là ghi rõ trong báo cáo đây là lỗi được cài đặt có chủ đích để minh họa)
- Trao đổi với giảng viên nếu muốn đề xuất chủ đề khác ngoài 5 gợi ý trên
- Làm việc nhóm nhỏ (nếu được giảng viên cho phép theo quy định cụ thể của lớp) với phân công rõ ràng

Tiêu chí đánh giá (liên hệ LO.03, LO.04)

Tiêu chí	Trọng số	Chuẩn đầu ra
Chất lượng mô hình hóa/phân tích	30%	LO.03
Kỹ thuật kiểm chứng/kiểm thử áp dụng đúng	30%	LO.04
Kết quả cụ thể + giải thích rõ ràng	25%	LO.03, LO.04
Chất lượng báo cáo + mã nguồn	15%	LO.04

Hỏi: Tôi có bắt buộc phải dùng CBMC/AFL thật không, hay dùng module Python trong toolkit là đủ?

Đáp: Dùng module Python trong softsec-toolkit là hoàn toàn đủ và được khuyến khích vì đã quen thuộc xuyên suốt học phần. Dùng thêm công cụ thật (CBMC, AFL) là điểm cộng nhưng không bắt buộc.

Hỏi: Chương trình mục tiêu có bắt buộc viết bằng C không?

Đáp: Không bắt buộc. Chương 2, 3 tập trung vào C vì đó là nơi lỗi bộ nhớ phổ biến nhất, nhưng Chủ đề 1 (Kripke+BMC) và Chủ đề 4 (Fuzzing) có thể áp dụng cho chương trình Python hoặc ngôn ngữ khác.

Hỏi: Nếu công cụ không tìm thấy lỗi nào trong chương trình của tôi thì sao?

Đáp: Đây vẫn là kết quả hợp lệ và có giá trị – miễn là bạn giải thích được *tại sao* không có lỗi (đã đạt completeness threshold? đã fuzzing đủ lâu với coverage cao?). Kiểm chứng “an toàn” một cách có căn cứ cũng là một kỹ năng quan trọng đã học.

Bước tiếp theo

- Xem chi tiết yêu cầu định dạng và cấu trúc báo cáo trong tài liệu *"Hướng dẫn Báo cáo Bài tập lớn"* (deck riêng)
- Đăng ký đề tài với giảng viên theo lịch trình học phần
- Bắt đầu ngay từ tuần đầu tiên – tránh dồn công việc vào giai đoạn cuối

Chúc các bạn thực hiện tốt Bài tập lớn!

Câu hỏi?